

动态内存

1. 操作系统中内存的一种形式
 1. 栈空间:局部变量、值传递参数
 2. 堆空间:动态内存分配的位置
 2. C:早在C之中已经有malloc和free等对动态内存操纵的函数。
 1. malloc() – memory allocation
 2. free() – free memory
 3. C++
 1. new – create space for a new object (allocate)
 2. delete – delete this object (free)
- [1. 动态对象](#)
 - [2. 创建对象](#)
 - [3. 对象的删除](#)
 - [4. 动态对象数组](#)
 - [5. 动态2D数组](#)

1. 动态对象

1. 在heap中创建
2. new/delete(constructor/destructor), 可以被重载
3. 为什么要引入new和delete操作符:因为新的操作符可以解决初始化函数的析构函数的调用的问题
4. 具体示例如下:

```
1 class A {
2     public :
3         A () ;
4         A (int);
5 };
6 A *p,*q;
7 p = new A;
8 //在程序的 heap 中申请一块大小为 sizeof(A) 的 内存
9 //新的功能:调用 A 的默认构造函数对该空间上的对象初始化
10 //返回创建的地址并赋值给 p
11 q = new A(1);
12 //调用 A 的另一个构造函数 A::A(int)
13 delete p;
14 //新功能:调用 p 所指 向 的对象的析构函数
15 //释放对象空间 delete q ;
```

1. malloc(不调用构造函数)|free(不调用析构函数)
 - new可以重载

```
1 p = (A *)malloc(sizeof(A))//A中的成员变量没有初始化
2 free(p)
```

2. 创建对象

1. new:

1. 使用原始类型
2. 使用类类型

2. Syntax:语法

1. 原始类型: `type* ptrName = new type;`
2. 使用类类型: `type* ptrName = new type(params);`

3. 注意:这是没有变量名字的物体



关于对象指针

3. 对象的删除

1. delete:

1. 唤起指向物体的指针
2. 处理原始类型或类类型

2. 语法: `delete ptrName;`

3. 注意:删除之后, 要将指针置为空指针, 这样子之后可以继续使用, 避免意外的引用对象,如果指针没有修改的话, 可能是一个悬挂指针(有可能出现段错误等等)

```
1 delete ptrName;  
2 ptrName = NULL;
```

4. 动态对象数组

1. 动态对象数组的创建与撤销

```
1 A *p;  
2 p = new A[100];  
3 delete []p;
```

2. 注意:

1. 不能显式初始化, 相应的类必须有默认构造函数
2. 初始化部分是修改比较多的

3. 在堆上分配的内存默认不进行初始化

1. `int *p1 = new int[5];` 默认不进行初始化
2. `int *p2 = new int[5]();` 进行默认初始化
3. `int *p2 = new int[5]{0,1,2,3,4};` 进行显式对应函数初始化

4. 注意: `delete []p` 中的 `[]` 不可以省略

1. 如果省略的话, 是删除了数组的第一个元素。并且会破坏其中的存储数组长度
2. `new int[100]` 就可以直接delete, 因为不是复杂对象

5. 动态2D数组

1. 创建算法:

1. 分配行的数量
2. 对于每一行分配列



```
1  const int ROWS = 3;
2  const int COLUMNS = 4;
3  char **chArray2;
4
5  // allocate the rows 粉色部分
6  chArray2 = new char* [ROWS];
7
8  // allocate the (pointer) elements for each row 蓝色部分
9  for (int row = 0; row < ROWS; row++ )
10     chArray2[row] = new char[ COLUMNS ];
```

1. 删除算法:和创建算法相反

```
1  for (int row = 0; row < ROWS; row++) {
2      delete []chArray2[ row ];
3      chArray2[ row ] = NULL;
4  }
5  delete []chArray2;
6  chArray2 = NULL;
```